

Company Internal Marketplace & Community Platform

Developer-ready Product, Functional & High-Level Architecture Specification

A private company platform where employees browse administrator-managed categories, create posts in any active category, interact with posts, and request new categories. The category framework is generic enough for Buy/Sell, Rentals, Services, Events, Lost & Found, Carpooling, Housing and other internal communities.

Users: Employee and Administrator/Moderator

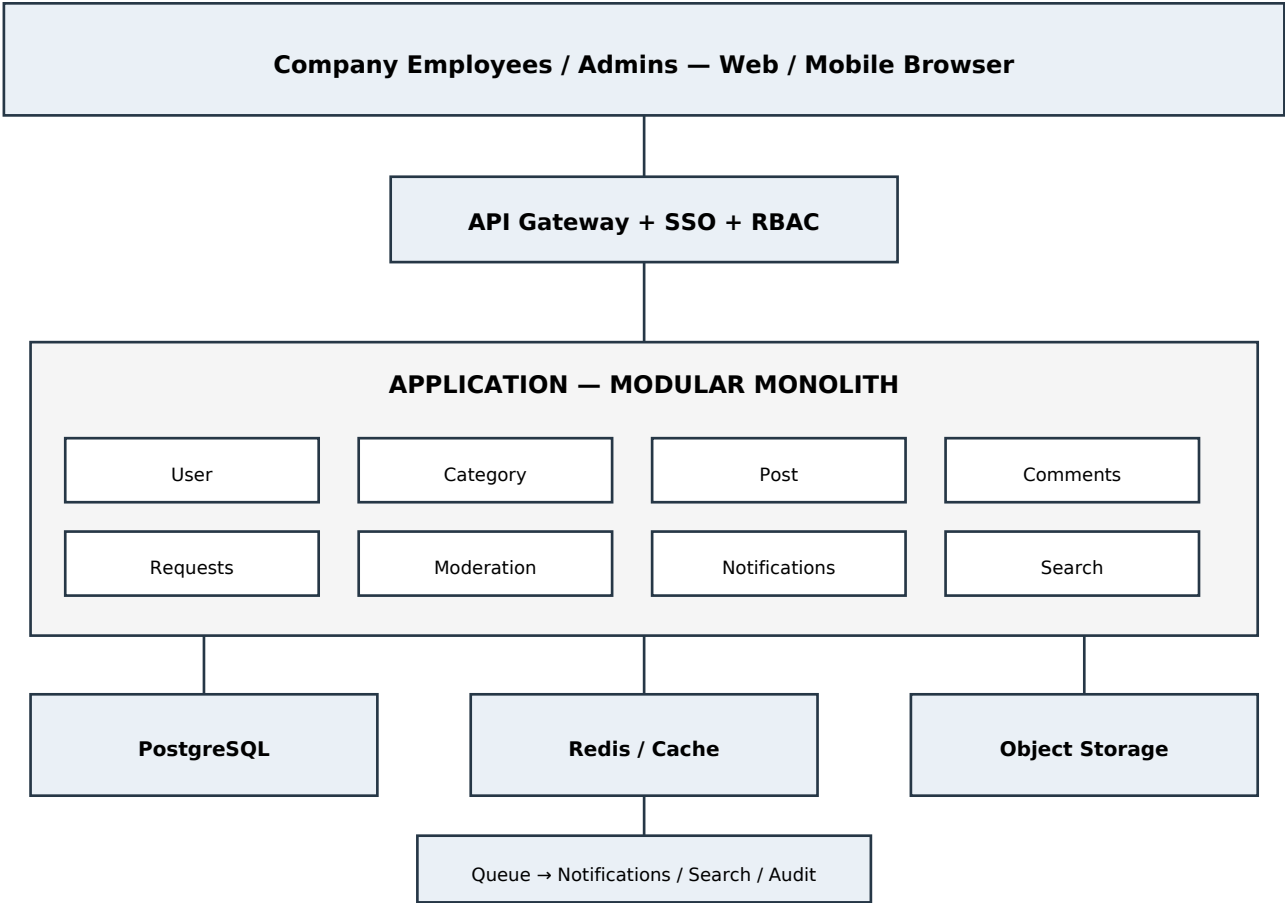
Objective: Give a development team enough product and technical detail to build the MVP without re-interpreting the product concept.

1. Functional Requirements

- FR-01 Authentication: Employees authenticate using corporate SSO.
- FR-02 User: Store employee identity and basic profile information.
- FR-03 Categories: Admin can create, edit, activate, deactivate and archive categories.
- FR-04 Browse: Employees can view all active categories and browse their posts.
- FR-05 Create post: Employee can post in every active category.
- FR-06 Post data: Title, description, post type, optional price, images/attachments and optional metadata.
- FR-07 Lifecycle: ACTIVE, CLOSED/SOLD, REMOVED, REJECTED, ARCHIVED; DRAFT may be added.
- FR-08 Ownership: Users can edit/close/delete their own posts; admins can moderate any post.
- FR-09 Category request: Employee can request a category that does not exist.
- FR-10 Approval: Admin can approve/reject requests and the requester is informed.
- FR-11 Comments: Employees can comment on posts; admins can moderate comments.
- FR-12 Search: Search title, description and category with pagination.
- FR-13 Notifications: Notify users about category decisions and relevant activity.
- FR-14 Reporting: Users can report inappropriate posts/comments; admins review reports.
- FR-15 Audit: Record important admin and moderation actions.
- FR-16 Responsive UI: Desktop and mobile browser support.

2. High-Level Architecture

Recommended first implementation: a modular monolith. Keep modules separated in code but deploy one backend initially. Individual modules can later become services if scale requires it.



Recommended infrastructure: PostgreSQL as system of record; object storage for images/files; Redis for caching/rate limiting where useful; a managed queue for asynchronous work. OpenSearch/Elasticsearch is optional when PostgreSQL search becomes insufficient.

3. Roles & Permissions

Capability	Employee	Admin
View active categories	✓	✓

Create post	✓	✓
Edit/close own post	✓	✓
Comment/report	✓	✓
Request category	✓	✓
Manage categories	—	✓
Approve/reject category request	—	✓
Moderate content	—	✓
Review reports	—	✓
View audit logs	—	✓

4. Core User Flows

Login: Employee → Corporate SSO → Identity Provider → Application identity → User record → RBAC → Home.

Browse: Home → Categories → Select category → Paginated post list → Post detail.

Create post: Select category → Form → Validate active category + fields → Upload files → Save → Confirmation.

Request category: Employee → Request New Category → Proposed name + reason → PENDING → Admin queue → Approve/Reject → Notification → Approved request creates/activates category.

Moderation: Post/comment/report → Validation → Active or review state → Admin action → State update → Audit → Optional notification.

Close listing: Owner → Mark SOLD/CLOSED → Listing becomes non-actionable according to policy.

5. Data Model

Entity	Important fields
User	id, employee_id, email, name, department, role, status, created_at, updated_at
Category	id, name, description, status, created_by, created_at, updated_at
CategoryRequest	id, requested_by, proposed_name, reason, status, reviewed_by, review_note, timestamps
Post	id, category_id, user_id, title, description, post_type, price, status, timestamps
PostImage	id, post_id, object_key, sort_order, metadata, created_at
Comment	id, post_id, user_id, text, status, timestamps
Report	id, reporter_id, target_type, target_id, reason, status, reviewed_by, timestamps
Notification	id, user_id, type, reference_id, message, read_at, created_at
AuditLog	id, actor_id, action, entity_type, entity_id, metadata, created_at

6. Category Specification

- Fields: name, description, status, created_by, timestamps. Optional future fields: icon, sort order, custom fields.
- Status values: ACTIVE, INACTIVE, ARCHIVED.
- Category names should be unique among active categories.
- Inactive categories cannot receive new posts.
- Frontend must load categories from API; category names must never be hard-coded.
- Prevent duplicate category requests when a matching active/pending category exists.

7. Post Specification

- Required: category, title, description.
- Optional: price, images, office/site, tags, expiry date and contact preference.
- Extensible post types: SELL, BUY_REQUEST, RENT, OFFER, REQUEST, INFORMATION.
- Server-side validation is mandatory.
- Owner can edit/close own posts; admin can moderate all posts.
- All feed/search endpoints must paginate.

- Images/files are stored in object storage; database stores references and metadata.

8. Suggested REST API

Area	Endpoint	Purpose
User	GET /api/v1/me	Current employee
Categories	GET /api/v1/categories	Active categories/search
Categories	POST /api/v1/admin/categories	Create category
Categories	PATCH /api/v1/admin/categories/{id}	Edit/status
Requests	POST /api/v1/category-requests	Request category
Requests	GET /api/v1/admin/category-requests	Admin queue
Requests	POST /api/v1/admin/category-requests/{id}/approve	Approve
Requests	POST /api/v1/admin/category-requests/{id}/reject	Reject
Posts	GET /api/v1/posts	Browse/search
Posts	POST /api/v1/posts	Create
Posts	GET /api/v1/posts/{id}	Details
Posts	PATCH /api/v1/posts/{id}	Edit own
Posts	DELETE /api/v1/posts/{id}	Remove own
Comments	GET/POST /api/v1/posts/{id}/comments	List/add
Reports	POST /api/v1/reports	Report content
Notifications	GET /api/v1/notifications	Notifications
Notifications	POST /api/v1/notifications/{id}/read	Mark read

9. Required Screens

- Login / SSO
- Home — search, categories, recent posts, create-post CTA
- Category page — description, filters and post list
- Post detail — images, description, price, owner, comments, report
- Create/Edit post
- My Posts — active/closed/archived
- Request New Category — form and request status
- Admin Dashboard — category management, pending requests, reports and moderation
- Notifications

10. Security & Non-Functional Requirements

- Use corporate OIDC/OAuth/SSO. Do not create a separate password system unless required.
- Enforce RBAC on backend APIs; UI hiding is not authorization.
- Validate/sanitize input; protect against SQL injection, XSS and unsafe HTML.
- Validate file type/size; generate safe object keys; use private/signed access where appropriate.
- Minimize employee information exposed to other employees.
- Rate-limit post creation, comments, reports, search and sensitive endpoints.
- Use structured logs, metrics, health checks and centralized error tracking.
- Backend should be stateless for horizontal scaling.
- Back up PostgreSQL and test restoration.
- Audit privileged and moderation actions.

11. Asynchronous Events

Event	Action
CATEGORY_REQUESTED	Notify admins
CATEGORY_APPROVED	Notify requester; optionally index

Event	Action
CATEGORY_REJECTED	Notify requester
POST_CREATED	Search indexing; optional notifications
POST_REPORTED	Create/notify moderation case
POST_MODERATED	Notify owner when appropriate; audit
COMMENT_CREATED	Notify post owner when appropriate

12. MVP Acceptance Criteria

- Employee can authenticate with company SSO.
- Admin can create a category without code deployment.
- New active category automatically appears to employees.
- Employee can create a post in every active category.
- Inactive/nonexistent categories reject new posts.
- Posts can be browsed/searched with pagination.
- Owners can edit and close their posts.
- Employees can request categories.
- Admins can approve/reject requests and requester receives result.
- Users can report content and admins can act on reports.
- Important admin/moderation actions are auditable.
- Images are stored outside PostgreSQL.
- Protected APIs reject unauthorized direct calls.
- Application is usable on desktop and mobile.

13. Delivery Plan

Phase 1: SSO, users, RBAC, database, category CRUD and base UI.

Phase 2: Posts, images, category feeds, search, post detail and close/sold.

Phase 3: Comments, category requests and notifications.

Phase 4: Reports, moderation, audit logs, security hardening and observability.

Phase 5: Advanced search, custom category fields, follows, recommendations and analytics.

14. Key Architectural Decisions

- Start as a modular monolith; split services only when scale or team boundaries justify it.
- Use corporate SSO rather than application-managed passwords.
- PostgreSQL is the system of record.
- Object storage holds uploaded files.
- Categories are dynamic database entities, never hard-coded.
- Authorization is enforced server-side.
- Use async events for notifications, indexing and other non-blocking work.
- Keep audit records for privileged/moderation actions.
- Use pagination, validation, stable IDs and idempotency where retries are possible.
- Keep the domain generic so new community types do not require redesign.

Definition of Done: MVP acceptance criteria pass, critical authorization/lifecycle paths have automated tests, deployment/backup/monitoring are documented, and an administrator can operate categories and moderation without developer intervention.